

ANATOMY-AWARE LOW-DOSE CT DENOISING

Complete Code-Generation Prompt & Implementation Guide

Revised & Clarified Edition — With DA-CLIP Explanation & Single-Dose Simplification

Target Hardware	NVIDIA RTX A5000 (24 GB VRAM)
CUDA Version	13.0 (backward compatible with cu121 PyTorch builds)
Driver	580.126.09
OS	Ubuntu 24 (remote via SSH)
Training Dataset	Mayo_2020_Simulated — 25% dose paired with 100% (HDCT)
Test Patient	L506 from Mayo_2016 (.IMA format)
Primary Goal	Train LDCT→NDCT denoiser at 25% dose, then infer on L506

TABLE OF CONTENTS

SECTION 0 — Clarifications — DA-CLIP, Single-Dose, TotalSegmentator 2D

SECTION 1 — Environment Setup

SECTION 2 — Dataset Structure, Preprocessing & Data Loading

SECTION 3 — Stage 1: VM-UNet Teacher (VSS Block, BYOL, Full Architecture)

SECTION 4 — Stage 2: Anatomy-Conditioned VSSD Denoiser

SECTION 5 — Diffusion Process (Residual Diffusion / RDDM)

SECTION 6 — Loss Functions (Progressive Schedule)

SECTION 7 — Training Scripts (Stage 1 & Stage 2)

SECTION 8 — Training Tips from FoundDiff Paper

SECTION 9 — Inference on Patient L506 (Mayo_2016)

SECTION 10 — Evaluation Metrics

SECTION 11 — Configuration Files

SECTION 12 — Common Issues & Fixes

SECTION 13 — Quick-Start Checklist

SECTION 0 — CRITICAL CLARIFICATIONS (READ FIRST)

Before diving into code, this section answers your direct questions about DA-CLIP, dose conditioning, TotalSegmentator with 2D images, and architecture simplifications made to ensure the code actually runs on your hardware.

0.1 Why DA-CLIP Exists — And Why You DON'T Need It For 25% Only

What DA-CLIP is for in FoundDiff: FoundDiff was designed to generalize across 8 different dose levels (1/10, 1/6, 1/4, 1/3, 1/2, 2/3, 1, etc.) in a single model. The core problem it solves: when you train on 8 dose levels simultaneously, the model needs to KNOW which dose level it is currently seeing so it can apply the right denoising strategy. A 10% dose image looks completely different from a 75% dose image. DA-CLIP is the component that reads the LDCT image and outputs a 'dose embedding' that tells the denoiser 'this is a 25% dose image, denoise accordingly.'

Why you don't need it for single-dose training: You are training ONLY on 25% dose images. Every LDCT in your training set is at 25% dose. The model never sees any other dose level during training. Therefore, there is no ambiguity about what dose level the input is — it's always 25%. The dose embedding would just be a constant vector for every sample, carrying zero useful information.

NOTE: If you later want to train on multiple dose levels (5%, 10%, 25%, 50% simultaneously), THEN you should bring in the full DA-CLIP with the ranking loss `L_rank`. For now, replace it with a simple learned constant vector or a tiny CNN.

0.2 DA-CLIP Replacement for Single-Dose Training

There are three options, in order of simplicity:

- **Option A (Recommended — Simplest):** Remove dose conditioning entirely. The timestep embedding alone is sufficient when training on a single dose. Set `e_d = 0` (zero tensor). `adaLN-Zero` then conditions only on timestep `t`.
- **Option B (Minimal):** Use a small learned constant embedding. A single `nn.Parameter` of shape `[1, 256]` that gets broadcast to `[B, 256]`. The model learns this as 'this is what 25% dose means' — but since it never changes, it's equivalent to a learned bias term.
- **Option C (Extensible):** Keep the `SimpleDoseEncoder` CNN (from Section 4.5 of the generated prompt), but do NOT use the full `open_clip` DA-CLIP. The CNN version reads the LDCT image and produces a 256-dim embedding. This is extensible: when you add more dose levels later, the same CNN will automatically adapt without architecture changes.

TIP: Use Option A first to get the model running. Switch to Option C before multi-dose training.

```
# Option A: No dose conditioning (simplest for single-dose)
class NoDoseEncoder(nn.Module):
    def __init__(self, out_dim=256):
        super().__init__()
        self.out_dim = out_dim
    def forward(self, x_ldct):
        return torch.zeros(x_ldct.shape[0], self.out_dim, device=x_ldct.device)

# Option B: Learned constant dose embedding
class ConstantDoseEncoder(nn.Module):
    def __init__(self, out_dim=256):
        super().__init__()
        self.dose_emb = nn.Parameter(torch.randn(1, out_dim))
    def forward(self, x_ldct):
```

```

        return self.dose_emb.expand(x_ldct.shape[0], -1)

# Option C: Simple CNN (recommended — extensible to multi-dose later)
class SimpleDoseEncoder(nn.Module):
    def __init__(self, out_dim=256):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Conv2d(1, 32, 7, stride=2, padding=3), # 256x256
            nn.SiLU(),
            nn.Conv2d(32, 64, 3, stride=2, padding=1), # 128x128
            nn.SiLU(),
            nn.Conv2d(64, 128, 3, stride=2, padding=1), # 64x64
            nn.SiLU(),
            nn.Conv2d(128, 256, 3, stride=2, padding=1), # 32x32
            nn.SiLU(),
            nn.AdaptiveAvgPool2d(1),
            nn.Flatten(),
            nn.Linear(256, out_dim)
        )
    def forward(self, x_ldct):
        return self.encoder(x_ldct) # [B, out_dim]

```

0.3 How TotalSegmentator Works With Your 2D Slices — Clear Answer

This is the most important pipeline question. Here is the exact flow:

Step	What Happens	Format
1	Load all .dcm/.npy slices for one patient	[N, 512, 512] float32
2	Stack slices into a 3D volume	[512, 512, N] — NIfTI convention
3	Un-normalize back to HU values	HU: air=-1000, bone=+1000
4	Save as NIfTI (.nii.gz) with affine matrix	TotalSegmentator reads this
5	Run TotalSegmentator on the 3D NIfTI	[512, 512, N] integer labels 0-103
6	Remap 104 classes → 7 denoising classes	[512, 512, N] labels 0-6
7	Split 3D mask → 2D per-slice .npy files	[512, 512] uint8, one per slice
8	During training: load mask alongside LDCT/HDCT pair	torch.long tensor [512, 512]

NOTE: TotalSegmentator NEVER sees individual 2D slices. It only sees the full 3D NIfTI volume. The 2D-to-3D stacking is YOUR job (done in run_total_segmentator.py, once per patient). After this preprocessing, training only loads the saved 2D .npy mask files — no 3D operations happen at training time.

Why run on HDCT (100%) only, never on LDCT (25%)? The patient does not move between LDCT and HDCT acquisitions — they are the SAME scan session, just reconstructed at different dose levels. So HDCT and LDCT slices are pixel-perfectly aligned. Running TotalSegmentator on noisy LDCT gives worse organ boundaries. Run on clean HDCT once, reuse the same mask for the paired LDCT slice.

0.4 Architecture Simplifications Made for Runnability

Component	Simplification & Reason
VSSD → SS2D	VSSD (Mamba2/SSD kernel) is harder to install and less stable. SS2D (Mamba1 selective scan) with a pure-
Full DA-CLIP removed	Replaced with SimpleDoseEncoder CNN (Option C above). Full DA-CLIP requires fine-tuning a ViT-B/32 CLIP
512x512 → 256x256 patches	Training on full 512x512 with batch size 8 may OOM on 24GB VRAM. Training on 256x256 random crops is s

L_rank removed initially	The multi-dose ranking loss requires CLIP-IQA and multiple dose levels simultaneously. Skip for single-dose
L_anatomy delayed to step 150k	This runs Stage 1 twice per step — expensive. It's applied every 5 steps after step 150k to keep training tract

SECTION 1 — ENVIRONMENT SETUP

Your machine: NVIDIA RTX A5000 (24 GB VRAM, Ampere = sm_86), CUDA 13.0, Ubuntu 24, SSH remote.

1.1 Conda Environment

```
# Create environment — Python 3.10 is best for Mamba/VMamba compatibility
conda create -n ldct_denoise python=3.10 -y
conda activate ldct_denoise

# PyTorch with CUDA 12.x (CUDA 13.0 is backward compatible with cu121 builds)
pip install torch==2.3.1 torchvision==0.18.1 --index-url https://download.pytorch.org/whl/cu121

# Verify GPU
python -c "import torch; print(torch.cuda.is_available(), torch.cuda.get_device_name(0))"
# Expected: True NVIDIA RTX A5000
```

1.2 Core Dependencies

```
# Medical imaging
pip install SimpleITK nibabel pydicom

# TotalSegmentator (for pseudo-label generation, run ONCE before training)
pip install TotalSegmentator
pip install nnunetv2 # if TotalSegmentator fails to pull it

# Image processing & metrics
pip install scikit-image opencv-python-headless einops timm

# Training utilities
pip install wandb tqdm pyyaml omegaconf

# General scientific
pip install numpy scipy matplotlib pandas h5py
```

1.3 VMamba / Mamba SSM Installation (May Fail — Read Carefully)

WARNING: The mamba_ssm CUDA kernel is the trickiest dependency. It compiles custom CUDA code during install. If it fails, DO NOT panic — the code includes a pure-Python fallback that is functionally identical (just 3-5x slower). You can train successfully with the fallback.

```
# Set architecture flag BEFORE installing (RTX A5000 = Ampere = sm_86)
export TORCH_CUDA_ARCH_LIST="8.6"

# Try mamba_ssm (attempt in this order):
pip install mamba-ssm==1.2.0
# If that fails:
pip install mamba-ssm==1.1.4
# If still fails:
pip install mamba-ssm --no-build-isolation

# causal-conv1d (dependency)
pip install causal-conv1d>=1.1.0

# Clone VMamba for VSS block reference (optional but useful)
git clone https://github.com/MzeroMiko/VMamba.git
cd VMamba && pip install -e . --no-build-isolation && cd ..

# If mamba_ssm fails entirely, the code automatically uses Python fallback.
```

```
# Set these to disable CUDA kernel attempt:  
export MAMBA_FORCE_BUILD=0
```

1.4 Useful Environment Variables (add to ~/.bashrc)

```
export CUDA_VISIBLE_DEVICES=0  
export TORCH_CUDA_ARCH_LIST="8.6"  
export PYTHONPATH=/path/to/ldct_project:$PYTHONPATH
```

SECTION 2 — DATASET STRUCTURE & LOADING

2.1 Your Complete Dataset Structure

```
data/                                     ← Your data root
├── Mayo_2016/
│   ├── HDCT/                             ← Full-dose CT, .IMA format (Siemens DICOM)
│   │   ├── L506/                         ← Patient ID folders
│   │   │   ├── slice_001.IMA
│   │   │   ├── ...
│   │   │   └── [other patients]/
│   │   └── LDCT/                         ← 25% dose, .IMA format
│   │       ├── L506/
│   │       └── [other patients]/
│   └── Mayo_2020/
│       ├── HDCT/                         ← Full-dose only, .dcm format
│       │   └── [patient_id]/
│       └── Mayo_2020_Simulated/          ← *** USE THIS FOR TRAINING ***
│           ├── [patient_id]/
│           │   ├── 5%/ 10%/ 20%/ 25%/    ← Simulated LDCT at different doses
│           │   ├── 50%/ 75%/
│           │   └── 100%/                  ← Same as Mayo_2020/HDCT (ground truth)
```

TRAINING PAIRS:

```
HDCT = Mayo_2020_Simulated/[patient]/100%/*.dcm (ground truth)
LDCT = Mayo_2020_Simulated/[patient]/25%/*.dcm  (input, 25% dose)
MASKS = Generated by TotalSegmentator on HDCT volumes (run_total_segmentator.py)
```

INFERENCE:

```
Patient L506 in Mayo_2016/LDCT/ and Mayo_2016/HDCT/ (.IMA format)
Convert .IMA to .npy using the same preprocessing script (it handles both .dcm and .IMA)
```

2.2 Answer: Do You Need to Run on HDCT Only or Both?

For TotalSegmentator: HDCT only. Run it once on the 100% dose volumes. The LDCT at 25% dose uses the SAME mask (same patient, same anatomy, only noise differs).

For training the denoiser (Stage 2): You need BOTH — the LDCT (25%) as input and the HDCT (100%) as ground truth. The dataset class loads triplets: (ldct_slice, hdct_slice, mask_slice).

For training the teacher (Stage 1): HDCT only. The segmentation teacher learns anatomy from clean images. During Stage 2 training, the frozen Stage 1 also receives LDCT as input (to generate conditioning signals for the noisy input) — but its weights were trained on HDCT.

2.3 Preprocessing Script — convert_ima_to_npy.py

NOTE: Run this ONCE before any training. Converts all .IMA/.dcm files to normalized .npy arrays. Only process 25% and 100% dose folders for initial training.

```
# data/preprocess/convert_ima_to_npy.py
import pydicom, numpy as np, os
from pathlib import Path

def read_dicom_slice(filepath):
    ds = pydicom.dcmread(str(filepath), force=True) # force=True handles .IMA
    pixel_array = ds.pixel_array.astype(np.float32)
```



```

    slope = float(getattr(ds, 'RescaleSlope', 1))
    intercept = float(getattr(ds, 'RescaleIntercept', 0))
    return pixel_array * slope + intercept # returns HU values

def normalize_hu(hu_array, window_center=40, window_width=400):
    """Soft tissue window: WC=40, WW=400 → HU range [-160, 240] → normalized [-1,1]"""
    hu_min = window_center - window_width / 2 # -160
    hu_max = window_center + window_width / 2 # 240
    clipped = np.clip(hu_array, hu_min, hu_max)
    normalized = (clipped - hu_min) / (hu_max - hu_min) # [0, 1]
    return (normalized * 2 - 1).astype(np.float32) # [-1, 1]

def process_patient_folder(patient_dir, output_dir):
    patient_dir, output_dir = Path(patient_dir), Path(output_dir)
    output_dir.mkdir(parents=True, exist_ok=True)
    extensions = ['.IMA', '.dcm', '.ima']
    files = []
    for ext in extensions:
        files.extend(patient_dir.glob(f'*{ext}'))
    if not files:
        print(f" WARNING: No DICOM files in {patient_dir}"); return
    slices = []
    for f in files:
        try:
            ds = pydicom.dcmread(str(f), stop_before_pixels=True, force=True)
            instance_num = int(getattr(ds, 'InstanceNumber', 0))
            slices.append((instance_num, f))
        except Exception as e:
            print(f" Could not read {f}: {e}")
    slices.sort(key=lambda x: x[0])
    for idx, (_, filepath) in enumerate(slices):
        hu = read_dicom_slice(filepath)
        norm = normalize_hu(hu)
        np.save(str(output_dir / f"slice_{idx:04d}.npy"), norm)
    print(f" Saved {len(slices)} slices to {output_dir}")

# ■■ Run for Mayo_2020_Simulated (training data) ■■
DATA_ROOT = "/path/to/data/Mayo_2020_Simulated"
OUTPUT_ROOT = "/path/to/data/Mayo_2020_Simulated_npy"
for patient in sorted(os.listdir(DATA_ROOT)):
    for dose in ['25%', '100%']: # Only these two for initial training
        src = Path(DATA_ROOT) / patient / dose
        dst = Path(OUTPUT_ROOT) / patient / dose
        if src.exists():
            print(f"Processing {patient}/{dose}...")
            process_patient_folder(src, dst)

# ■■ Also run for Mayo_2016 (test data, L506) ■■
MAYO2016_ROOT = "/path/to/data/Mayo_2016"
MAYO2016_NPY = "/path/to/data/Mayo_2016_npy"
for split_folder in ['HDCT', 'LDCT']:
    for patient in sorted(os.listdir(Path(MAYO2016_ROOT) / split_folder)):
        src = Path(MAYO2016_ROOT) / split_folder / patient
        dst = Path(MAYO2016_NPY) / split_folder / patient
        if src.exists():
            process_patient_folder(src, dst)

```

2.4 TotalSegmentator Pseudo-Label Generation

NOTE: This script stacks 2D slices → 3D NIfTI → runs TotalSegmentator → saves per-slice 2D mask .npy. Run ONCE per patient on the 100% dose (HDCT) folder only.

```
# data/preprocess/run_total_segmentator.py
import numpy as np, nibabel as nib, subprocess, os
from pathlib import Path

# Mapping from TotalSegmentator v2 integer labels → 7 denoising classes
# Run 'totalsegmentator --help' or check TS source to verify exact indices for your TS version
TS_INT_TO_7CLASS = {
    # 0 = background (default, stays 0)
    1: 1, # spleen → class 1 (solid abdominal organs)
    2: 2, # kidney_right → class 2 (kidneys)
    3: 2, # kidney_left → class 2
    5: 1, # liver → class 1
    10: 3, # aorta → class 3 (large vessels)
    11: 3, # inferior_vena_cava → class 3
    # Lungs (TS labels ~50-60): → class 4
    # Vertebrae (TS labels ~80-95): → class 5 (bone)
    # Ribs (~96-120): → class 5
    # Everything else unmapped → class 6 (soft tissue/muscle)
}

# NOTE: Generate FULL mapping by running TS once and printing:
# import json; d=json.load(open('totalsegmentator/data/class_map_v2.json'))
# print(d) → maps integer → organ name

def remap_ts_mask(ts_mask):
    out = np.zeros_like(ts_mask)
    for ts_label, class7 in TS_INT_TO_7CLASS.items():
        out[ts_mask == ts_label] = class7
    unmapped = (out == 0) & (ts_mask > 0)
    out[unmapped & (ts_mask >= 79)] = 5 # likely bone
    out[unmapped & (ts_mask < 79) & (ts_mask > 0)] = 6 # soft tissue
    return out.astype(np.uint8)

def generate_masks_for_patient(hdct_npy_dir, output_mask_dir, tmp_dir="/tmp/ts_work"):
    hdct_dir, out_dir = Path(hdct_npy_dir), Path(output_mask_dir)
    out_dir.mkdir(parents=True, exist_ok=True)
    tmp = Path(tmp_dir); tmp.mkdir(parents=True, exist_ok=True)
    npy_files = sorted(hdct_dir.glob("slice*.npy"))
    if not npy_files: print(f"No slices in {hdct_dir}"); return
    # Stack 2D → 3D
    slices = [np.load(str(f)) for f in npy_files]
    volume = np.stack(slices, axis=0) # [N, 512, 512]
    # Un-normalize [-1,1] back to HU for TotalSegmentator
    volume_hu = ((volume + 1) / 2 * 400 + (-160)).astype(np.float32)
    # Create NIfTI (TS expects [H, W, N] with voxel spacing)
    affine = np.diag([1.0, 1.0, 3.0, 1.0]) # 1mm in-plane, ~3mm slice thickness
    nifti_vol = np.transpose(volume_hu, (1, 2, 0)) # [512, 512, N]
    nib.save(nib.Nifti1Image(nifti_vol, affine), str(tmp / "input.nii.gz"))
    # Run TotalSegmentator
    result = subprocess.run([
        "TotalSegmentator", "-i", str(tmp/"input.nii.gz"),
        "-o", str(tmp/"ts_output"), "--ml"
    ], capture_output=True, text=True)
    if result.returncode != 0:
        print(f" ERROR: {result.stderr}"); return
    mask_nifti = nib.load(str(tmp/"ts_output/total.nii.gz"))
    mask_3d = np.transpose(np.array(mask_nifti.dataobj), (2, 0, 1)) # [N, 512, 512]
    mask_7 = remap_ts_mask(mask_3d)
    for idx, npy_file in enumerate(npy_files):
```

```

        np.save(str(out_dir / f"mask_{idx:04d}.npy"), mask_7[idx])
    print(f" Saved {len(npy_files)} masks to {out_dir}")

if __name__ == "__main__":
    DATA_NPY = "/path/to/Mayo_2020_Simulated_npy"
    MASK_DIR = "/path/to/Mayo_2020_Simulated_masks"
    for patient in sorted(os.listdir(DATA_NPY)):
        hdct_dir = Path(DATA_NPY) / patient / "100%"
        if hdct_dir.exists():
            generate_masks_for_patient(hdct_dir, Path(MASK_DIR) / patient)

```

2.5 PyTorch Dataset Class

```

# data/dataset.py
import torch, numpy as np, os
from torch.utils.data import Dataset
from pathlib import Path

class LDCTDataset(Dataset):
    def __init__(self, data_root, mask_root, dose='25%', split='train',
                 val_patients=None, patch_size=256):
        self.data_root = Path(data_root)
        self.mask_root = Path(mask_root)
        self.dose = dose
        self.patch_size = patch_size
        if val_patients is None:
            val_patients = [] # set 1-2 patient IDs aside for validation
        self.samples = []
        for patient in sorted(os.listdir(data_root)):
            is_val = (patient in val_patients)
            if split == 'train' and is_val: continue
            if split == 'val' and not is_val: continue
            ldct_dir = self.data_root / patient / dose
            hdct_dir = self.data_root / patient / '100%'
            mask_dir = self.mask_root / patient
            if not (ldct_dir.exists() and hdct_dir.exists() and mask_dir.exists()):
                continue
            ldct_files = sorted(ldct_dir.glob('slice_*.npy'))
            hdct_files = sorted(hdct_dir.glob('slice_*.npy'))
            mask_files = sorted(mask_dir.glob('mask_*.npy'))
            n = min(len(ldct_files), len(hdct_files), len(mask_files))
            for i in range(n):
                self.samples.append((str(ldct_files[i]),
                                    str(hdct_files[i]),
                                    str(mask_files[i])))
            print(f"Dataset ({split}, {dose}): {len(self.samples)} slices")

    def __len__(self): return len(self.samples)

    def __getitem__(self, idx):
        ldct_path, hdct_path, mask_path = self.samples[idx]
        ldct = np.load(ldct_path) # [512, 512] float32 in [-1,1]
        hdct = np.load(hdct_path)
        mask = np.load(mask_path) # [512, 512] uint8 values 0-6
        ldct = torch.from_numpy(ldct).unsqueeze(0) # [1, 512, 512]
        hdct = torch.from_numpy(hdct).unsqueeze(0)
        mask = torch.from_numpy(mask.astype(np.int64)) # [512, 512]
        # Random crop to patch_size (reduces memory, standard practice)
        if self.patch_size is not None:
            h, w = ldct.shape[-2:]

```

```
top = torch.randint(0, h - self.patch_size + 1, (1,)).item()
left = torch.randint(0, w - self.patch_size + 1, (1,)).item()
p = self.patch_size
ldct = ldct[:, top:top+p, left:left+p]
hdct = hdct[:, top:top+p, left:left+p]
mask = mask[top:top+p, left:left+p]
return {'ldct': ldct, 'hdct': hdct, 'mask': mask}
```

SECTION 3 — STAGE 1: VM-UNET TEACHER NETWORK

Stage 1 is a VM-UNet trained to segment organs in CT images. It is trained first, then frozen. Its outputs (S = soft segmentation map, e_a = anatomy embeddings) are used as conditioning signals for Stage 2 during every training step.

3.1 VSS Block — Core Building Block (with Python Fallback)

```
# models/vss_block.py
import torch, torch.nn as nn, torch.nn.functional as F
import math

try:
    from mamba_ssm.ops.selective_scan_interface import selective_scan_fn
    MAMBA_AVAILABLE = True
    print("[OK] Fast CUDA mamba_ssm loaded")
except ImportError:
    MAMBA_AVAILABLE = False
    print("[WARN] mamba_ssm not available – using pure-Python fallback (3-5x slower, correct)")

def selective_scan_python_fallback(u, delta, A, B, C, D=None, z=None):
    """Pure-Python selective scan. Correct but slow. u: [B,L,D]"""
    B_batch, L, D_dim = u.shape
    N = A.shape[1]
    delta_A = torch.exp(delta.unsqueeze(-1) * A) # [B,L,D,N]
    delta_B_u = delta.unsqueeze(-1) * B.unsqueeze(2) * u.unsqueeze(-1) # [B,L,D,N]
    x = torch.zeros(B_batch, D_dim, N, device=u.device, dtype=u.dtype)
    ys = []
    for i in range(L):
        x = delta_A[:, i] * x + delta_B_u[:, i] # [B, D, N]
        y = (x * C[:, i].unsqueeze(1)).sum(-1) # [B, D]
        ys.append(y)
    y = torch.stack(ys, dim=1) # [B, L, D]
    if D is not None:
        y = y + u * D.unsqueeze(0).unsqueeze(0)
    if z is not None:
        y = y * F.silu(z)
    return y

class SS2D(nn.Module):
    """2D Selective Scan – scans in 4 directions (→ ← ↓ ↑). Input/output: [B,H,W,C]"""
    def __init__(self, d_model, d_state=16, d_conv=3, expand=2, dropout=0.0):
        super().__init__()
        self.d_model = d_model
        self.d_state = d_state
        self.d_inner = int(expand * d_model)
        self.in_proj = nn.Linear(d_model, self.d_inner * 2, bias=False)
        self.conv2d = nn.Conv2d(self.d_inner, self.d_inner, kernel_size=d_conv,
                                padding=d_conv//2, groups=self.d_inner, bias=True)
        self.x_proj_weight = nn.Parameter(
            torch.randn(4, self.d_inner, self.d_state * 2 + self.d_inner))
        self.dt_projs_weight = nn.Parameter(torch.randn(4, self.d_inner, self.d_inner))
        self.dt_projs_bias = nn.Parameter(torch.zeros(4, self.d_inner))
        A = torch.arange(1, self.d_state+1).float().repeat(self.d_inner, 1)
        self.A_logs = nn.Parameter(torch.log(A))
        self.Ds = nn.Parameter(torch.ones(self.d_inner))
        self.out_proj = nn.Linear(self.d_inner, d_model, bias=False)
        self.dropout = nn.Dropout(dropout) if dropout > 0 else nn.Identity()
```

```

def forward(self, x):
    B, H, W, C = x.shape
    xz = self.in_proj(x)
    x_main, z = xz.chunk(2, dim=-1)
    x_conv = F.silu(self.conv2d(x_main.permute(0, 3, 1, 2))).permute(0, 2, 3, 1)
    xs = [
        x_conv.reshape(B, H*W, -1),
        x_conv.flip([1, 2]).reshape(B, H*W, -1),
        x_conv.permute(0, 2, 1, 3).reshape(B, H*W, -1),
        x_conv.permute(0, 2, 1, 3).flip([1, 2]).reshape(B, H*W, -1),
    ]
    A = -torch.exp(self.A_logs) # [d_inner, d_state]
    ys = []
    for i, x_seq in enumerate(xs):
        x_dbl = x_seq @ self.x_proj_weight[i].T
        dt, B_ssm, C_ssm = x_dbl.split(
            [self.d_inner, self.d_state, self.d_state], dim=-1)
        dt = F.softplus(dt @ self.dt_projs_weight[i].T + self.dt_projs_bias[i])
        if MAMBA_AVAILABLE:
            y = selective_scan_fn(
                x_seq.transpose(1, 2), dt.transpose(1, 2),
                A, B_ssm.transpose(1, 2), C_ssm.transpose(1, 2), self.Ds
            ).transpose(1, 2)
        else:
            y = selective_scan_python_fallback(x_seq, dt, A, B_ssm, C_ssm, self.Ds)
        ys.append(y)
    y = (
        ys[0].reshape(B, H, W, -1) +
        ys[1].reshape(B, H, W, -1).flip([1, 2]) +
        ys[2].reshape(B, H, W, -1).permute(0, 2, 1, 3) +
        ys[3].reshape(B, H, W, -1).flip([1, 2]).permute(0, 2, 1, 3)
    )
    y = y * F.silu(z)
    return self.dropout(self.out_proj(y))

class VSSBlock(nn.Module):
    def __init__(self, hidden_dim, d_state=16, expand=2, dropout=0.0):
        super().__init__()
        self.ln = nn.LayerNorm(hidden_dim)
        self.ss2d = SS2D(hidden_dim, d_state=d_state, expand=expand, dropout=dropout)
        self.ls = nn.Parameter(torch.ones(hidden_dim) * 1e-5) # LayerScale

    def forward(self, x): # x: [B, H, W, C]
        return x + self.ls * self.ss2d(self.ln(x))

```

3.2 PatchMerging & PatchExpanding

```

class PatchMerging(nn.Module):
    """2x spatial downsampling: [B,C,H,W] → [B,2C,H/2,W/2]"""
    def __init__(self, dim):
        super().__init__()
        self.norm = nn.LayerNorm(4 * dim)
        self.reduction = nn.Linear(4 * dim, 2 * dim, bias=False)
    def forward(self, x):
        B, C, H, W = x.shape
        x = x.permute(0, 2, 3, 1) # [B,H,W,C]
        x0=x[:,0::2,0::2,:]; x1=x[:,1::2,0::2,:];
        x2=x[:,0::2,1::2,:]; x3=x[:,1::2,1::2,:];
        x = self.norm(torch.cat([x0,x1,x2,x3], dim=-1))
        return self.reduction(x).permute(0, 3, 1, 2) # [B,2C,H/2,W/2]

```

```

class PatchExpanding(nn.Module):
    """2x spatial upsampling: [B,C,H,W] → [B,C/2,2H,2W]"""
    def __init__(self, dim):
        super().__init__()
        self.norm = nn.LayerNorm(dim)
        self.expand = nn.Linear(dim, 2 * dim, bias=False)
    def forward(self, x):
        B, C, H, W = x.shape
        x = self.expand(self.norm(x.permute(0,2,3,1))) # [B,H,W,2C]
        x = x.reshape(B,H,W,2,2,C//2).permute(0,1,3,2,4,5).reshape(B,H*2,W*2,C//2)
        return x.permute(0, 3, 1, 2) # [B,C/2,2H,2W]

```

3.3 Full VM-UNet (Stage 1 Teacher)

Takes a CT image, outputs S (organ probability map), e_a (per-organ feature vectors), and F (bottleneck features for BYOL). Architecture: 4-level UNet with VSS blocks.

```

# models/vm_unet.py
import torch, torch.nn as nn, torch.nn.functional as F

class VMUNet(nn.Module):
    """
    VM-UNet Stage 1 teacher.
    Input: [B, 1, H, W] – single-channel CT slice
    Outputs: S [B,7,H,W], e_a [B,7,C], F [B,768,H/32,W/32]
    """
    def __init__(self, in_chans=1, num_classes=7, dims=[96,192,384,768],
                  depths=[2,2,2,2], d_state=16):
        super().__init__()
        self.num_classes = num_classes
        # Patch embedding: 4x4 stride-4 → 96 channels at 1/4 resolution
        self.patch_embed = nn.Conv2d(in_chans, dims[0], kernel_size=4, stride=4)
        self.patch_norm = nn.LayerNorm(dims[0])
        # Encoder: 3 stages + bottleneck
        self.encoder_blocks = nn.ModuleList([
            nn.ModuleList([VSSBlock(dims[i], d_state=d_state) for _ in range(depths[i])])
            for i in range(4)
        ])
        self.downsamplers = nn.ModuleList([PatchMerging(dims[i]) for i in range(3)])
        # Decoder: 3 stages
        self.upsamplers = nn.ModuleList([PatchExpanding(dims[3-i]) for i in range(3)])
        self.skip_projs = nn.ModuleList([
            nn.Conv2d(dims[2-i]*2, dims[2-i], 1) for i in range(3)
        ])
        self.decoder_blocks = nn.ModuleList([
            nn.ModuleList([VSSBlock(dims[2-i], d_state=d_state) for _ in range(depths[i])])
            for i in range(3)
        ])
        self.final_upsample = nn.Upsample(scale_factor=4, mode='bilinear', align_corners=False)
        self.seg_head = nn.Conv2d(dims[0], num_classes, 1)

    def _run_enc_blocks(self, x, blocks):
        x_bhwc = x.permute(0,2,3,1)
        for blk in blocks:
            x_bhwc = blk(x_bhwc)
        return x_bhwc.permute(0,3,1,2)

    def forward(self, x):
        B, _, H, W = x.shape
        # Patch embed

```

```

x = self.patch_embed(x) # [B, 96, H/4, W/4]
x = x.permute(0,2,3,1); x = self.patch_norm(x); x = x.permute(0,3,1,2)
# Encoder
skips = []
for i in range(3):
    x = self._run_enc_blocks(x, self.encoder_blocks[i])
    skips.append(x)
    x = self.downsamplers[i](x)
# Bottleneck
x = self._run_enc_blocks(x, self.encoder_blocks[3])
F_bottleneck = x # [B, 768, H/32, W/32]
# Decoder
for i in range(3):
    x = self.upsamplers[i](x)
    skip = skips[2-i]
    x = self.skip_projs[i](torch.cat([x, skip], dim=1))
    x = self._run_dec_blocks(x, self.decoder_blocks[i])
decoder_feats = x # [B, 96, H/4, W/4]
x = self.final_upsample(x)
logits = self.seg_head(x) # [B, 7, H, W]
S = torch.softmax(logits, dim=1)
# Masked average pooling for anatomy embeddings
S_dec = F.interpolate(S, size=decoder_feats.shape[-2:],
                      mode='bilinear', align_corners=False)
e_a = self._masked_avg_pool(decoder_feats, S_dec) # [B, 7, 96]
return {'logits': logits, 'S': S, 'e_a': e_a, 'F': F_bottleneck}

def _masked_avg_pool(self, features, S):
    """features: [B,C,H,W], S: [B,K,H,W] → [B,K,C]"""
    e_a = []
    for k in range(S.shape[1]):
        w = S[:, k] # [B, H, W]
        weighted = (w.unsqueeze(1) * features).sum(dim=[2,3])
        norm = w.sum(dim=[1,2]).unsqueeze(1).clamp(min=1e-6)
        e_a.append(weighted / norm)
    return torch.stack(e_a, dim=1)

```

3.4 BYOL for Noise Invariance

NOTE: BYOL forces Stage 1's bottleneck features to be invariant to dose level. When Stage 2 feeds LDCT (noisy) into frozen Stage 1 during training, the features are consistent with what Stage 1 saw during its own training. Weight starts at 0 (first 5 epochs), then 0.1.

```

# models/byol.py
class BYOLProjector(nn.Module):
    def __init__(self, in_dim=768, hidden_dim=4096, out_dim=256):
        super().__init__()
        self.net = nn.Sequential(
            nn.AdaptiveAvgPool2d(1), nn.Flatten(),
            nn.Linear(in_dim, hidden_dim), nn.BatchNorm1d(hidden_dim), nn.ReLU(),
            nn.Linear(hidden_dim, out_dim))
    def forward(self, x): return self.net(x)

class BYOLPredictor(nn.Module):
    def __init__(self, dim=256, hidden=4096):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(dim, hidden), nn.BatchNorm1d(hidden), nn.ReLU(),
            nn.Linear(hidden, dim))
    def forward(self, x): return self.net(x)

```



```
def byol_loss(q, z_target):
    q = F.normalize(q, dim=-1)
    z = F.normalize(z_target.detach(), dim=-1)
    return 2 - 2 * (q * z).sum(dim=-1).mean()

# EMA update (call after each optimizer step):
# for p_on, p_tgt in zip(online_proj.parameters(), target_proj.parameters()):
#     p_tgt.data = 0.996 * p_tgt.data + 0.004 * p_on.data
```

SECTION 4 — STAGE 2: ANATOMY-CONDITIONED DENOISER

4.1 Spatial FiLM Module

FiLM (Feature-wise Linear Modulation) applies per-pixel scale/shift to feature maps based on the organ segmentation map S. This is the 'where are the organs?' signal. Zero-initialized so it starts as identity and learns modulation gradually.

```
# models/spatial_film.py
import torch, torch.nn as nn, torch.nn.functional as F

class SpatialFiLM(nn.Module):
    """Per-pixel organ-aware feature modulation using soft segmentation map S."""
    def __init__(self, num_classes=7, hidden_size=64):
        super().__init__()
        self.film_net = nn.Sequential(
            nn.Conv2d(num_classes, 32, 3, padding=1), nn.SiLU(),
            nn.Conv2d(32, 64, 3, padding=1), nn.SiLU(),
            nn.Conv2d(64, 2 * hidden_size, 1))
        # Zero-init → identity at start of training
        nn.init.zeros_(self.film_net[-1].weight)
        nn.init.zeros_(self.film_net[-1].bias)

    def forward(self, x, S_feat):
        """x: [B,C,H,W], S_feat: [B,7,H,W] → [B,C,H,W]"""
        film_params = self.film_net(S_feat)
        gamma, beta = film_params.chunk(2, dim=1)
        x_norm = F.layer_norm(x, x.shape[1:])
        return (1 + gamma) * x_norm + beta # identity when gamma=0, beta=0
```

4.2 Cross-Attention With Anatomy Embeddings

Each spatial position in the feature map queries the 7 organ embeddings (e_a) to ask: 'what do my nearby organs look like in this specific patient?' This is the 'what do the organs look like?' signal, complementing FiLM's spatial 'where' signal.

```
# models/cross_attention.py
class CrossAttentionAnatomy(nn.Module):
    def __init__(self, hidden_size, anatomy_dim=96, num_heads=None):
        super().__init__()
        if num_heads is None:
            num_heads = max(1, hidden_size // 32)
        # Project e_a to match hidden_size if needed
        self.kv_proj = nn.Linear(anatomy_dim, hidden_size) if anatomy_dim != hidden_size else nn.Identity()
        self.attn = nn.MultiheadAttention(
            embed_dim=hidden_size, num_heads=num_heads, batch_first=True)
        self.norm = nn.LayerNorm(hidden_size)

    def forward(self, x, e_a):
        """x: [B,C,H,W], e_a: [B,7,C_a] → [B,C,H,W]"""
        B, C, H, W = x.shape
        x_flat = x.permute(0,2,3,1).reshape(B, H*W, C)
        e_a_proj = self.kv_proj(e_a) # [B, 7, C]
        attended, _ = self.attn(x_flat, e_a_proj, e_a_proj)
        out = self.norm(x_flat + attended)
        return out.reshape(B,H,W,C).permute(0,3,1,2)
```

4.3 adaLN-Zero (Timestep + Dose Conditioning)

```

class AdaLNZero(nn.Module):
    """Adaptive LayerNorm: conditions normalization on timestep+dose. Zero-initialized."""
    def __init__(self, hidden_size, cond_dim):
        super().__init__()
        self.norm = nn.LayerNorm(hidden_size, elementwise_affine=False)
        self.proj = nn.Linear(cond_dim, 2 * hidden_size)
        nn.init.zeros_(self.proj.weight); nn.init.zeros_(self.proj.bias)

    def forward(self, x, cond):
        """x: [B,C,H,W], cond: [B,cond_dim] → [B,C,H,W]"""
        B, C, H, W = x.shape
        x_norm = self.norm(x.permute(0,2,3,1)) # [B,H,W,C]
        gamma, beta = self.proj(cond).chunk(2, dim=-1) # each [B,C]
        gamma = gamma[:,None,None,:]; beta = beta[:,None,None,:]
        return ((1+gamma)*x_norm + beta).permute(0,3,1,2)

```

4.4 AnatomyMamba Block (Core of Stage 2)

Each block in the denoising UNet applies: (1) adaLN-Zero for timestep conditioning, (2) Spatial FiLM for organ-location conditioning, (3) SS2D scan for global context, (4) Cross-attention for semantic organ information. All with residual connections.

```

# models/anatomy_mamba_block.py
import math

def sinusoidal_embedding(t, dim):
    half = dim // 2
    freqs = torch.exp(-math.log(10000) * torch.arange(half, device=t.device) / half)
    args = t[:,None].float() * freqs[None]
    return torch.cat([torch.cos(args), torch.sin(args)], dim=-1)

class TimestepEmbedder(nn.Module):
    def __init__(self, dim):
        super().__init__()
        self.mlp = nn.Sequential(
            nn.Linear(dim, dim*4), nn.SiLU(), nn.Linear(dim*4, dim*4))
        self.dim = dim
    def forward(self, t):
        return self.mlp(sinusoidal_embedding(t, self.dim))

class AnatomyMambaBlock(nn.Module):
    def __init__(self, hidden_size, cond_dim, anatomy_embed_dim=96, num_classes=7, d_state=16):
        super().__init__()
        self.adaln = AdaLNZero(hidden_size, cond_dim)
        self.spatial_film = SpatialFiLM(num_classes, hidden_size)
        self.ss2d = SS2D(hidden_size, d_state=d_state)
        self.cross_attn = CrossAttentionAnatomy(hidden_size, anatomy_embed_dim)
        self.out_proj = nn.Conv2d(hidden_size, hidden_size, 1)

    def forward(self, x, S_feat, e_a, t_cond):
        """
        x:          [B, C, H, W]   feature map
        S_feat:     [B, 7, H, W]   segmentation at this resolution
        e_a:        [B, 7, C_a]    anatomy embeddings (from Stage 1)
        t_cond:     [B, cond_dim]  timestep + dose embedding
        """
        residual = x
        x = self.adaln(x, t_cond) # timestep conditioning
        x = self.spatial_film(x, S_feat) # organ-location conditioning
        x_bhwc = self.ss2d(x.permute(0,2,3,1)) # global scan

```

```

x = x_bhwc.permute(0,3,1,2)
x = self.cross_attn(x, e_a)           # semantic organ lookup
return residual + self.out_proj(x)   # residual connection

```

4.5 Dose Encoder (Simplified — Use This for 25% Training)

```

# models/da_clip.py
# For single-dose (25%) training: use Option A (no dose) or Option C (simple CNN)
# See Section 0.2 for full explanation.

class SimpleDoseEncoder(nn.Module):
    """
    Simple CNN dose encoder. Sufficient for single-dose training.
    Extensible: add more dose levels later without architecture changes.
    Replace with full DA-CLIP only when training on 3+ dose levels simultaneously.
    """
    def __init__(self, out_dim=256):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Conv2d(1, 32, 7, stride=2, padding=3), nn.SiLU(),
            nn.Conv2d(32, 64, 3, stride=2, padding=1), nn.SiLU(),
            nn.Conv2d(64, 128, 3, stride=2, padding=1), nn.SiLU(),
            nn.Conv2d(128, 256, 3, stride=2, padding=1), nn.SiLU(),
            nn.AdaptiveAvgPool2d(1), nn.Flatten(),
            nn.Linear(256, out_dim))
    def forward(self, x_ldct): return self.encoder(x_ldct)

```

4.6 Full Stage 2 UNet

```

# models/unet_denoiser.py
class AnatomyUNet(nn.Module):
    """
    Stage 2 denoising UNet.
    Input:  noisy_residual [B,1,H,W] + ldct [B,1,H,W] (concatenated = 2 channels)
    Output: predicted_residual [B,1,H,W], seg_kd_logits [B,7,H,W]
    """
    def __init__(self, base_dim=64, dim_mults=(1,2,4,8),
                 num_classes=7, anatomy_embed_dim=96):
        super().__init__()
        dims = [base_dim * m for m in dim_mults] # [64,128,256,512]
        cond_dim = base_dim * 4 # 256
        self.init_conv = nn.Conv2d(2, dims[0], 3, padding=1)
        self.time_embed = TimestepEmbedder(base_dim)
        self.dose_enc = SimpleDoseEncoder(out_dim=cond_dim)
        self.cond_proj = nn.Sequential(
            nn.Linear(base_dim*4, cond_dim), nn.SiLU(), nn.Linear(cond_dim, cond_dim))
        # Downsampling
        self.down_blocks = nn.ModuleList()
        self.downsamplers = nn.ModuleList()
        for i, d in enumerate(dims):
            self.down_blocks.append(
                AnatomyMambaBlock(d, cond_dim, anatomy_embed_dim, num_classes))
            if i < len(dims)-1:
                self.downsamplers.append(nn.Conv2d(d, dims[i+1], 2, stride=2))
        # Bottleneck
        self.bottleneck = AnatomyMambaBlock(dims[-1], cond_dim, anatomy_embed_dim, num_classes)
        # Seg KD head (attached to bottleneck features)
        self.seg_kd_head = nn.Sequential(
            nn.ConvTranspose2d(dims[-1], dims[-1]//2, 4, stride=2, padding=1), nn.ReLU(),
            nn.ConvTranspose2d(dims[-1]//2, num_classes, 4, stride=2, padding=1))

```

```

# Upsampling
self.up_blocks = nn.ModuleList()
self.upsamplers = nn.ModuleList()
reversed_dims = list(reversed(dims))
for i, d in enumerate(reversed_dims):
    in_d = d + dims[len(dims)-1-i] if i > 0 else d
    block_dim = reversed_dims[i]
    self.up_blocks.append(
        AnatomyMambaBlock(block_dim, cond_dim, anatomy_embed_dim, num_classes))
    if i < len(dims)-1:
        next_d = reversed_dims[i+1]
        self.upsamplers.append(nn.Sequential(
            nn.Upsample(scale_factor=2, mode='bilinear', align_corners=False),
            nn.Conv2d(block_dim, next_d, 3, padding=1)))
self.final_norm = nn.GroupNorm(8, dims[0])
self.final_conv = nn.Conv2d(dims[0], 1, 1)

def forward(self, x_noisy, t, x_ldct, S_scales, e_a):
    """
    x_noisy: [B,1,H,W]   noisy residual at timestep t
    t:       [B]         diffusion timestep
    x_ldct:  [B,1,H,W]   LDCT (self-conditioning)
    S_scales: list[Tensor] segmentation at [H/4,H/8,H/16,H/32]
    e_a:     [B,7,96]    anatomy embeddings from frozen Stage 1
    """
    t_emb = self.time_embed(t) # [B, base_dim*4]
    d_emb = self.dose_enc(x_ldct) # [B, cond_dim]
    cond = self.cond_proj(t_emb) + d_emb # [B, cond_dim]
    x = self.init_conv(torch.cat([x_noisy, x_ldct], dim=1))
    skips = []
    for i, block in enumerate(self.down_blocks):
        S_feat = F.interpolate(S_scales[0], size=x.shape[-2:],
                               mode='bilinear', align_corners=False)
        x = block(x, S_feat, e_a, cond)
        skips.append(x)
        if i < len(self.downsamplers):
            x = self.downsamplers[i](x)
    # Bottleneck
    S_bot = F.interpolate(S_scales[0], size=x.shape[-2:],
                          mode='bilinear', align_corners=False)
    x = self.bottleneck(x, S_bot, e_a, cond)
    bottleneck_feats = x
    seg_kd = F.interpolate(self.seg_kd_head(bottleneck_feats),
                           size=S_scales[0].shape[-2:], mode='bilinear', align_corners=False)
    # Upsampling
    for i, block in enumerate(self.up_blocks):
        if i < len(self.upsamplers):
            x = self.upsamplers[i](x)
        skip = skips[-(i+2)] if i < len(skips)-1 else skips[0]
        if x.shape == skip.shape:
            x = x + skip
        S_feat = F.interpolate(S_scales[0], size=x.shape[-2:],
                               mode='bilinear', align_corners=False)
        x = block(x, S_feat, e_a, cond)
    predicted_residual = self.final_conv(F.silu(self.final_norm(x)))
    return predicted_residual, seg_kd

```

SECTION 5 — DIFFUSION PROCESS (RESIDUAL DIFFUSION / RDDM)

Instead of diffusing the entire image, we diffuse only the RESIDUAL ($x_{ndct} - x_{ldct}$). The LDCT is mostly correct — most pixels are close to NDCT. The residual is a small correction. Diffusing a small signal is easier and faster than diffusing a full image.

```
# models/diffusion.py
import torch, numpy as np

class ResidualDiffusion:
    def __init__(self, T=1000, beta_schedule='cosine'):
        self.T = T
        betas = self._make_schedule(T, beta_schedule)
        alphas = 1 - betas
        alphas_bar = torch.cumprod(alphas, dim=0)
        self.betas = betas
        self.alphas_bar = alphas_bar
        self.sqrt_ab = alphas_bar.sqrt()
        self.sqrt_1mab = (1 - alphas_bar).sqrt()

    def _make_schedule(self, T, schedule):
        if schedule == 'cosine':
            x = torch.linspace(0, T, T+1)
            f = torch.cos(((x/T + 0.008)/1.008) * torch.pi/2) ** 2
            ab = f / f[0]
            betas = 1 - ab[1:] / ab[:-1]
            return betas.clamp(0, 0.999)
        return torch.linspace(1e-4, 0.02, T)

    def to(self, device):
        self.alphas_bar = self.alphas_bar.to(device)
        self.sqrt_ab = self.sqrt_ab.to(device)
        self.sqrt_1mab = self.sqrt_1mab.to(device)
        return self

    def q_sample(self, x0, t, noise=None):
        """Add t-level noise to residual x0. Returns (noisy, noise)."""
        if noise is None: noise = torch.randn_like(x0)
        s_ab = self.sqrt_ab[t].view(-1,1,1,1)
        s_1mab = self.sqrt_1mab[t].view(-1,1,1,1)
        return s_ab * x0 + s_1mab * noise, noise

    @torch.no_grad()
    def ddim_sample(self, model, x_ldct, S_scales, e_a, ddim_steps=50):
        """Reverse diffusion with DDIM. Returns predicted residual."""
        B, _, H, W = x_ldct.shape
        device = x_ldct.device
        x_t = torch.randn(B, 1, H, W, device=device)
        step_size = self.T // ddim_steps
        timesteps = list(range(0, self.T, step_size))[:-1]
        for t_val in timesteps:
            t = torch.full((B,), t_val, device=device, dtype=torch.long)
            pred_noise, _ = model(x_t, t, x_ldct, S_scales, e_a)
            ab_t = self.alphas_bar[t_val]
            ab_prev = self.alphas_bar[max(t_val - step_size, 0)]
            pred_x0 = (x_t - (1-ab_t).sqrt() * pred_noise) / ab_t.sqrt()
            pred_x0 = pred_x0.clamp(-1, 1)
            x_t = ab_prev.sqrt() * pred_x0 + (1-ab_prev).sqrt() * pred_noise
        return x_t
```

SECTION 6 — LOSS FUNCTIONS (PROGRESSIVE SCHEDULE)

Loss	Formula	Weight	Start Step	Purpose
L_res	MSE(pred_noise, noise)	1.0	0 (always)	Primary denoising — predict added noise
L_seg	CrossEntropy(logits, mask, smooth=0.1)	1.0	0 (Stage 1 only)	Segmentation teacher
L_byol	$2 - 2 \cdot \cos_{\text{sim}}(q, z_{\text{target}})$	0.1	Stage 1 epoch 5	Noise invariance of Stage 1 features
L_kd	CrossEntropy(seg_kd_logits, mask, smooth=0.1)	0.1	Step 50k	Stage 2 bottleneck anatomy awareness
L_anatomy	L1(e_a_pred, e_a_gt)	0.05	Step 150k (every 5 steps)	Anatomy preservation in denoised output

```
# losses/losses.py
import torch, torch.nn.functional as F

class StageTwoLosses:
    def __init__(self, lambda_kd=0.1, lambda_anatomy=0.05,
                 kd_start=50000, anatomy_start=150000):
        self.lambda_kd = lambda_kd; self.lambda_anatomy = lambda_anatomy
        self.kd_start = kd_start; self.anatomy_start = anatomy_start

    def l_res(self, pred_noise, true_noise):
        return F.mse_loss(pred_noise, true_noise)

    def l_kd(self, seg_pred, mask_labels):
        """Label-smoothed cross-entropy. Prevents overconfidence in pseudo-labels."""
        smooth, n = 0.1, seg_pred.shape[1]
        with torch.no_grad():
            dist = torch.full_like(seg_pred, smooth/(n-1))
            dist.scatter_(1, mask_labels.unsqueeze(1), 1.0 - smooth)
        return -(dist * F.log_softmax(seg_pred, dim=1)).sum(dim=1).mean()

    def l_anatomy(self, e_a_pred, e_a_gt):
        """L1 on per-organ feature vectors. Penalizes anatomy distortion."""
        return F.l1_loss(e_a_pred, e_a_gt)

    def compute(self, pred_noise, true_noise, seg_pred, mask_labels,
               e_a_pred=None, e_a_gt=None, step=0):
        losses = {}
        losses['l_res'] = self.l_res(pred_noise, true_noise)
        total = losses['l_res']
        if step >= self.kd_start and seg_pred is not None:
            losses['l_kd'] = self.l_kd(seg_pred, mask_labels)
            total = total + self.lambda_kd * losses['l_kd']
        if step >= self.anatomy_start and e_a_pred is not None:
            losses['l_anatomy'] = self.l_anatomy(e_a_pred, e_a_gt)
            total = total + self.lambda_anatomy * losses['l_anatomy']
        losses['total'] = total
        return losses
```

SECTION 7 — TRAINING SCRIPTS

7.1 Stage 1 Training Script

```
# train/train_stage1.py
import torch
from torch.utils.data import DataLoader
from torch.optim import AdamW
from torch.optim.lr_scheduler import CosineAnnealingLR

def train_stage1(config):
    device = torch.device('cuda')
    train_ds = LDCTDataset(config.data_root, config.mask_root,
                           dose='100%', split='train', patch_size=256)
    loader = DataLoader(train_ds, batch_size=8, shuffle=True,
                        num_workers=4, pin_memory=True)
    model = VMUNet(in_chans=1, num_classes=7).to(device)
    online_proj = BYOLProjector(768, 4096, 256).to(device)
    online_pred = BYOLPredictor(256).to(device)
    target_proj = BYOLProjector(768, 4096, 256).to(device)
    target_proj.load_state_dict(online_proj.state_dict())
    for p in target_proj.parameters(): p.requires_grad_(False)
    all_params = (list(model.parameters()) + list(online_proj.parameters())
                  + list(online_pred.parameters()))
    optimizer = AdamW(all_params, lr=1e-4, weight_decay=0.01)
    scheduler = CosineAnnealingLR(optimizer, T_max=100)
    ce_loss = torch.nn.CrossEntropyLoss(label_smoothing=0.1)

    for epoch in range(100):
        model.train(); byol_weight = 0.1 if epoch >= 5 else 0.0
        for batch in loader:
            hdct = batch['hdct'].to(device)
            ldct = batch['ldct'].to(device)
            mask = batch['mask'].to(device)
            out = model(hdct)
            l_seg = ce_loss(out['logits'], mask)
            if byol_weight > 0:
                x1 = hdct + 0.01 * torch.randn_like(hdct)
                x2 = ldct
                out1, out2 = model(x1), model(x2)
                z1, z2 = online_proj(out1['F']), online_proj(out2['F'])
                q1, q2 = online_pred(z1), online_pred(z2)
                with torch.no_grad():
                    z1_t = target_proj(model(x1)['F']).detach()
                    z2_t = target_proj(model(x2)['F']).detach()
                l_byol = byol_loss(q1, z2_t) + byol_loss(q2, z1_t)
            else:
                l_byol = torch.tensor(0., device=device)
            total = l_seg + byol_weight * l_byol
            optimizer.zero_grad(); total.backward()
            torch.nn.utils.clip_grad_norm_(all_params, 1.0)
            optimizer.step()
            if byol_weight > 0:
                for p_on, p_tgt in zip(online_proj.parameters(), target_proj.parameters()):
                    p_tgt.data = 0.996 * p_tgt.data + 0.004 * p_on.data
        scheduler.step()
    if (epoch+1) % 10 == 0:
        torch.save({'epoch': epoch, 'model': model.state_dict()},
                  f'checkpoints/stage1_epoch{epoch+1}.pt')
    print(f"Ep {epoch+1} | l_seg={l_seg.item():.4f} | l_byol={l_byol.item():.4f}")
```


7.2 Stage 2 Training Script

```
# train/train_stage2.py
import torch, torch.nn.functional as F
from torch.utils.data import DataLoader
from torch.optim import AdamW

def train_stage2(config):
    device = torch.device('cuda')
    # Load FROZEN Stage 1
    stage1 = VMUNet().to(device)
    stage1.load_state_dict(torch.load('checkpoints/stage1_epoch100.pt')['model'])
    stage1.eval()
    for p in stage1.parameters(): p.requires_grad_(False)
    print("Stage 1 loaded and frozen.")

    stage2 = AnatomyUNet(base_dim=64, dim_mults=(1,2,4,8)).to(device)
    diffusion = ResidualDiffusion(T=1000, beta_schedule='cosine').to(device)
    train_ds = LDCTDataset(config.data_root, config.mask_root,
                           dose='25%', split='train', patch_size=256)
    loader = DataLoader(train_ds, batch_size=8, shuffle=True,
                       num_workers=4, pin_memory=True)
    optimizer = AdamW(stage2.parameters(), lr=2e-5, weight_decay=0.01)
    losses_fn = StageTwoLosses()
    global_step = 0

    for epoch in range(200):
        stage2.train()
        for batch in loader:
            ldct = batch['ldct'].to(device) # [B,1,H,W]
            hdct = batch['hdct'].to(device)
            mask = batch['mask'].to(device)

            # STEP A: frozen Stage 1 → anatomy conditioning
            with torch.no_grad():
                s1_out = stage1(ldct)
                S = s1_out['S'] # [B,7,H,W]
                e_a = s1_out['e_a'] # [B,7,96]
                S_scales = [F.interpolate(S, size=sz, mode='bilinear', align_corners=False)
                           for sz in [(128,128),(64,64),(32,32),(16,16)]]

            # STEP B: diffusion forward
            true_res = hdct - ldct
            t = torch.randint(0, 1000, (ldct.shape[0],), device=device)
            noisy_res, noise = diffusion.q_sample(true_res, t)

            # STEP C: Stage 2 forward
            pred_noise, seg_kd = stage2(noisy_res, t, ldct, S_scales, e_a)

            # STEP D: L_anatomy (expensive – only every 5 steps after 150k)
            e_a_pred = e_a_gt = None
            if global_step >= 150000 and global_step % 5 == 0:
                ab = diffusion.sqrt_ab[t].view(-1,1,1,1)
                s1mab = diffusion.sqrt_1mab[t].view(-1,1,1,1)
                pred_res_hat = (noisy_res - s1mab * pred_noise) / ab.clamp(min=1e-6)
                x_hat = (ldct + pred_res_hat.detach()).clamp(-1, 1)
                with torch.no_grad():
                    e_a_pred = stage1(x_hat)['e_a']
                    e_a_gt = stage1(hdct)['e_a']

            # STEP E: losses
```

```

losses = losses_fn.compute(pred_noise, noise, seg_kd, mask,
                           e_a_pred, e_a_gt, step=global_step)
optimizer.zero_grad(); losses['total'].backward()
torch.nn.utils.clip_grad_norm_(stage2.parameters(), 1.0)
optimizer.step()
global_step += 1

if global_step % 500 == 0:
    print(f"Step {global_step} | " +
          " | ".join(f"{k}={v.item():.4f}" for k,v in losses.items()))
if (epoch+1) % 20 == 0:
    torch.save({'epoch': epoch, 'step': global_step,
               'model': stage2.state_dict()},
              f'checkpoints/stage2_epoch{epoch+1}.pt')

```

SECTION 8 — TRAINING TIPS FROM FOUNDDIFF PAPER

These are the exact training decisions used in FoundDiff, adapted for your setup:

Batch Size 8 / OOM Fix

FoundDiff used batch size 8 on V100 (32GB). Your A5000 has 24GB — may OOM at full 512x512. Fix: use `patch_size=256` (standard; FoundDiff also patches). If still OOM, reduce to `batch_size=4` and use `gradient_accumulation_steps=2` to simulate batch size 8. Also add `'with torch.cuda.amp.autocast()'` for fp16 — cuts memory ~40%.

Learning Rates

Stage 1 (DA-CLIP in their paper): $1e-4$, cosine decay, 100 epochs. Stage 2 (denoiser): $2e-5$, much smaller because the model is large and noisy gradients from diffusion require gentle updates. Never use $1e-4$ for Stage 2 — it diverges.

Gradient Clipping = 1.0

FoundDiff explicitly uses grad norm clipping at 1.0. Essential for Mamba models — the selective scan can produce large gradients at early training. Always clip.

T=1000 Diffusion Steps, DDIM at Inference

Train with $T=1000$ forward steps. FoundDiff samples with only 2 DDIM steps at inference for residual diffusion (fast because the residual is small). Start with 50 DDIM steps, reduce to 10 once training converges.

Image Window: [-160, 240] HU

FoundDiff normalizes CT to $[-1, 1]$ using soft tissue window: $WC=40$, $WW=400 \rightarrow$ HU range $[-160, 240]$. This is already in the preprocessing script. Consistent normalization is critical — ALL slices must use the same window.

Self-Conditioning (2-channel input)

Concatenate LDCT with noisy residual as 2-channel input to Stage 2. This is 'DA-Diff self-conditioning' from the paper. The model can see the clean LDCT at every step, which helps it stay grounded.

EMA of Model Weights

FoundDiff keeps an EMA copy of Stage 2 ($\tau=0.9999$) and uses it at evaluation. Implement: after each step, $\text{ema_params} = 0.9999 \cdot \text{ema_params} + 0.0001 \cdot \text{params}$. Use EMA model for PSNR/SSIM evaluation. Typically adds ~0.3 dB PSNR improvement.

Cosine Noise Schedule

FoundDiff uses cosine beta schedule (not linear). Cosine schedule has less noise at early and late timesteps, smoother gradient signal. Already implemented in the diffusion code.

Multi-Dose Ranking Loss (L_{rank}) — SKIP FOR NOW

FoundDiff trains on 8 dose levels simultaneously with a contrastive ranking loss to enforce quality ordering. You are training on 25% only — SKIP L_{rank} entirely. Add it only when you extend to multi-dose training.

FoundDiff Configuration Summary (from paper)

Parameter	DA-CLIP (Stage 1 equiv.)	DA-Diff (Stage 2)	Your Config
Epochs/Steps	100 epochs	400k steps	100 ep / 200k steps
Batch Size	192 (multi-GPU)	8	8 (reduce to 4 if OOM)
Learning Rate	1e-4	2e-5	Same
Image Size	256x256	256x256	256x256 patches
T (diffusion)	N/A	1000	1000
DDIM steps	N/A	2 at inference	50 → reduce to 2
Optimizer	Adam	Adam	AdamW (better)
Grad Clip	1.0	1.0	1.0
Noise Schedule	N/A	Cosine	Cosine

SECTION 9 — INFERENCE ON PATIENT L506 (MAYO_2016)

NOTE: Mayo_2016 uses .IMA format (Siemens DICOM). The same `convert_ima_to_npy.py` script handles this — pydicom with `force=True` reads .IMA files natively.

```
# inference/denoise_patient.py
import torch, numpy as np, torch.nn.functional as F
from pathlib import Path

def denoise_patient(patient_id='L506',
                    stage1_ckpt='checkpoints/stage1_epoch100.pt',
                    stage2_ckpt='checkpoints/stage2_epoch200.pt',
                    output_dir='results/L506',
                    ddim_steps=50):
    device = torch.device('cuda')
    Path(output_dir).mkdir(parents=True, exist_ok=True)

    # Load models
    stage1 = VMUNet().to(device).eval()
    stage2 = AnatomyUNet(base_dim=64, dim_mults=(1,2,4,8)).to(device).eval()
    stage1.load_state_dict(torch.load(stage1_ckpt)['model'])
    stage2.load_state_dict(torch.load(stage2_ckpt)['model'])
    for p in list(stage1.parameters()) + list(stage2.parameters()):
        p.requires_grad_(False)

    diffusion = ResidualDiffusion(T=1000).to(device)

    # Load L506 LDCT slices (preprocessed to .npy via convert_ima_to_npy.py)
    ldct_dir = Path(f"/path/to/Mayo_2016_npy/LDCT/{patient_id}")
    hdct_dir = Path(f"/path/to/Mayo_2016_npy/HDCT/{patient_id}")
    ldct_files = sorted(ldct_dir.glob("slice_*.npy"))

    results = []
    for i, f in enumerate(ldct_files):
        ldct_np = np.load(str(f)) # [512, 512]
        ldct_t = torch.from_numpy(ldct_np).unsqueeze(0).unsqueeze(0).to(device)

        with torch.no_grad():
            # Anatomy conditioning (Stage 1)
            s1_out = stage1(ldct_t)
            S, e_a = s1_out['S'], s1_out['e_a']
            S_scales = [F.interpolate(S, size=sz, mode='bilinear', align_corners=False)
                        for sz in [(128,128),(64,64),(32,32),(16,16)]]
            # DDIM reverse diffusion
            pred_res = diffusion.ddim_sample(stage2, ldct_t, S_scales, e_a, ddim_steps)
            denoised = (ldct_t + pred_res).clamp(-1, 1)

        denoised_np = denoised.squeeze().cpu().numpy()
        np.save(f"{output_dir}/denoised_{i:04d}.npy", denoised_np)
        if i % 20 == 0:
            print(f" Slice {i+1}/{len(ldct_files)}")

    # Compute metrics if HDCT ground truth is available
    if hdct_dir.exists():
        from evaluate.eval import compute_metrics
        hdct_files = sorted(hdct_dir.glob("slice_*.npy"))
        for i, (lf, hf) in enumerate(zip(ldct_files, hdct_files)):
            denoised_np = np.load(f"{output_dir}/denoised_{i:04d}.npy")
            hdct_np = np.load(str(hf))
            results.append(compute_metrics(denoised_np, hdct_np))
```

```
print(f"L506 PSNR={np.mean([r['psnr'] for r in results]):.2f} dB | "  
      f"SSIM={np.mean([r['ssim'] for r in results]):.4f}")  
print(f"Done. Results saved to {output_dir}")
```

SECTION 10 — EVALUATION METRICS

```
# evaluate/eval.py
import numpy as np
from skimage.metrics import peak_signal_noise_ratio, structural_similarity
from scipy.ndimage import sobel, binary_dilation

def compute_metrics(denoised, hdct, data_range=2.0):
    """denoised, hdct: [H,W] float32 in [-1,1]. data_range=2.0 for [-1,1] range."""
    psnr = peak_signal_noise_ratio(hdct, denoised, data_range=data_range)
    ssim = structural_similarity(hdct, denoised, data_range=data_range)
    return {'psnr': psnr, 'ssim': ssim}

def dice_score(pred_mask, gt_mask, class_id):
    pred = (pred_mask == class_id); gt = (gt_mask == class_id)
    intersection = (pred & gt).sum()
    total = pred.sum() + gt.sum()
    return 1.0 if total == 0 else float(2 * intersection) / float(total)

def anatomy_weighted_ssim(denoised, hdct, organ_mask):
    """SSIM with 3x weight on organ pixels (organ_mask: bool [H,W])."""
    ssim_map = structural_similarity(hdct, denoised, data_range=2.0, full=True)[1]
    weights = np.where(organ_mask, 3.0, 1.0)
    return float((ssim_map * weights).sum() / weights.sum())

def boundary_f1(denoised, hdct, threshold=0.1, tolerance=2):
    """F1 score measuring edge preservation quality."""
    def edges(img):
        mag = np.sqrt(sobel(img,0)**2 + sobel(img,1)**2)
        return mag > threshold
    pe, ge = edges(denoised), edges(hdct)
    ge_d = binary_dilation(ge, iterations=tolerance)
    pe_d = binary_dilation(pe, iterations=tolerance)
    tp = (pe & ge_d).sum(); fp = (pe & ~ge_d).sum(); fn = (ge & ~pe_d).sum()
    prec = tp / (tp + fp + 1e-8); rec = tp / (tp + fn + 1e-8)
    return float(2 * prec * rec / (prec + rec + 1e-8))
```

SECTION 11 — CONFIGURATION FILES

```
# configs/stage1_train.yaml
data_root:    /path/to/Mayo_2020_Simulated_npy
mask_root:    /path/to/Mayo_2020_Simulated_masks
dose:         '100%'          # Use HDCT for Stage 1 training
val_patients: []              # Add 1-2 patient IDs for validation
patch_size:   256             # 256x256 random crops; null for full 512x512
model:
  in_chans:    1
  num_classes: 7
  dims:        [96, 192, 384, 768]
  depths:      [2, 2, 2, 2]
  d_state:     16
training:
  epochs:      100
  batch_size:  8              # Reduce to 4 if OOM
  lr:          1.0e-4
  weight_decay: 0.01
  grad_clip:   1.0
  byol_start_ep: 5
  byol_weight: 0.1
  num_workers: 4
checkpoints:
  save_dir:    checkpoints/stage1
  save_every:  10             # epochs

---
# configs/stage2_train.yaml
data_root:    /path/to/Mayo_2020_Simulated_npy
mask_root:    /path/to/Mayo_2020_Simulated_masks
dose:         '25%'          # LDCT = 25% simulated dose
val_patients: []
patch_size:   256
model:
  base_dim:    64
  dim_mults:   [1, 2, 4, 8]
  anatomy_embed_dim: 96
  num_classes: 7
diffusion:
  T:           1000
  beta_schedule: cosine
  ddim_steps:  50             # at inference (reduce to 2-10 once working)
training:
  total_steps: 200000
  batch_size:  8
  lr:          2.0e-5
  weight_decay: 0.01
  grad_clip:   1.0
  num_workers: 4
  lambda_kd:   0.1
  lambda_anatomy: 0.05
  kd_start_step: 50000
  anatomy_start: 150000
stage1_ckpt:  checkpoints/stage1/stage1_epoch100.pt
checkpoints:
  save_dir:    checkpoints/stage2
  save_every:  10000         # steps
```


SECTION 12 — COMMON ISSUES & FIXES

Issue: CUDA Out of Memory (OOM)

1. Reduce `batch_size` 8 → 4. 2. Set `patch_size=256` (already recommended). 3. Wrap forward pass in `'with torch.cuda.amp.autocast():'` (fp16, cuts memory 40%). 4. Add gradient accumulation: accumulate for 2 steps before `optimizer.step()`.

Issue: mamba_ssm fails to compile

1. Export: `'export TORCH_CUDA_ARCH_LIST=8.6'` (RTX A5000 = Ampere = sm_86) BEFORE pip install. 2. Try: `'pip install mamba-ssm==1.1.4 --no-build-isolation'`. 3. If all fail: the code automatically uses pure-Python fallback. Set `MAMBA_AVAILABLE=False`.

Issue: TotalSegmentator fails

1. Ensure nibabel is installed: `pip install nibabel`. 2. Check NIfTI affine is valid (non-zero diagonal). 3. Try: `'--fast'` flag for faster (less accurate) segmentation. 4. `pip install nnunetv2==2.2` separately.

Issue: Diffusion loss doesn't decrease

1. Verify `true_residual = hdct - ldct` (not the other way). 2. Verify `q_sample` returns (noisy, noise) and you're computing `MSE(pred_noise, noise)`. 3. Check normalization: both `hdct` and `ldct` should be in `[-1,1]`.

Issue: Stage 1 Dice stays near 0

1. Print `mask.unique()` — should show `tensor([0,1,2,3,4,5,6])`. 2. If showing 0-103: TotalSegmentator remapping failed. Check `remap_ts_mask()`. 3. Verify HDCT normalization is consistent with training.

Issue: Cross-attention gives NaN

1. Check normalizer clamp: `.clamp(min=1e-6)` in `masked_avg_pool`. 2. Verify `num_heads` divides `hidden_size` evenly (`hidden_size // 32`). 3. Check `e_a` shape is `[B, 7, 96]` — not `[B, 96, 7]`.

Issue: .IMA files not loading

1. Use `pydicom.dcmread(path, force=True)` — the `force=True` flag handles .IMA. 2. `pip install pydicom` (if not installed). 3. Check file extension: `glob` both `'IMA'` and `'ima'` (case-sensitive on Linux).

Issue: Stage 2 training very slow

1. Stage 1 forward (for `S`, `e_a`) runs every step inside `torch.no_grad()` — this is expected. 2. `L_anatomy` also runs Stage 1 twice — but only every 5 steps after step 150k. 3. Use `num_workers=4` in `DataLoader` for parallel data loading. 4. Use `pin_memory=True` in `DataLoader` for faster CPU→GPU transfers.

SECTION 13 — QUICK-START CHECKLIST

Run steps in exact order. Do not skip preprocessing.

	Step 1	Set up conda environment, install PyTorch + core deps (Section 1)
		Attempt mamba_ssm install. If fails, verify Python fallback works (Section 1.3)
	Step 3	Run convert_ima_to_npy.py on Mayo_2020_Simulated (25% and 100% folders only)
		Run convert_ima_to_npy.py on Mayo_2016 HDCT/LDCT for L506 (test data)
	Step 5	Run run_total_segmentator.py on all 100% dose patients → generates mask .npy
		Run verify_dataset.py: check shapes (512,512), value range [-1,1], mask values [0-6]
	Step 7	Train Stage 1: python train/train_stage1.py --config configs/stage1_train.yaml
		Monitor Stage 1: Dice(liver) > 0.7 by epoch 50 = on track. Full convergence ~epoch 80-100
	Step 9	After epoch 100, freeze Stage 1. Begin Stage 2 training.
		Train Stage 2: python train/train_stage2.py --config configs/stage2_train.yaml
	Step 11	Monitor Stage 2: L_res should drop from ~1.0 to ~0.01 over first 50k steps
		At step 50k: L_kd activates. Expect small bump in total loss, then it decreases.
	Step 13	At step 150k: L_anatomy activates every 5 steps. Visual quality should improve.
		Run inference on L506: python inference/denoise_patient.py --patient L506
	Step 15	Evaluate: PSNR ~43-45 dB, SSIM ~0.96-0.98 is reasonable for 25% dose

Expected Results for 25% Dose Training on Mayo_2020_Simulated

Metric	Expected Range	Notes
PSNR (L506 test)	~43-46 dB	FoundDiff achieves ~45.94 dB at 1/10 dose with all components
SSIM	~0.96-0.98	Anatomy-weighted SSIM will be slightly lower
Dice (Liver)	> 0.80 after Stage 1	Should reach 0.85+ by Stage 1 epoch 100
L_res at step 200k	< 0.01	Primary convergence signal for Stage 2

Good luck! If specific sections are unclear, ask for clarification on that component.